DSC5001: Statistical Machine Learning I

Week 10: Ensemble Learning: Bagging, Random Forest, and Boosting

Kangning Cui
City University of Hong Kong (Dongguan)

Semester A, 2026/27

Ensemble Learning

DSC5001 ▪ Semester A, 2026/27

CART Review

In CART, a feature vector has q entries, $\boldsymbol{x} = (x_1, \ldots, x_q)^\top$, where q is the number of predictors.
Here $j \in \{1, \ldots, q\}$ selects a feature and s is its split threshold:

$$\boxed{x_j \leq s \quad \text{or} \quad x_j > s.}$$

Each terminal leaf predicts a class or a numerical mean.

Strength: local rules capture nonlinear patterns

Problem: a small data change may change those rules

This instability is the starting point for ensemble learning.

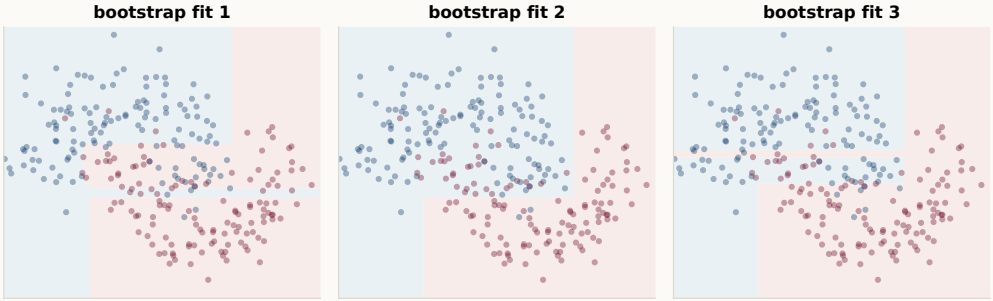
Delivery-Time Data

Consider six parcel deliveries. Each column is one training case i.

Route	A	B	C	D	E	F
Distance d_i (km)	1	2	3	4	5	6
Congestion c_i (0/1)	0	1	0	1	1	0
Rain w_i (0/1)	0	0	1	1	1	0
Time y_i (min)	12	14	17	22	26	29

Here $\boldsymbol{x}_i = (d_i, c_i, w_i)^\top$, y_i is the observed time, and $n = 6$.
Indicators $c_i = 1$ and $w_i = 1$ mean congestion and rain, respectively.
We will reuse these six records for Bagging, OOB, and Random Forest.

Tree Instability



Look at the boundaries: their locations differ across panels, so the same new input may receive different predictions. This is sampling instability.

Sampling Variability

At a fixed new input $\boldsymbol{x}_*$, trees trained on two hypothetical samples can disagree:

$$\widehat{f}_{\mathcal{D}_1}(\boldsymbol{x}_*) = 24, \qquad \widehat{f}_{\mathcal{D}_2}(\boldsymbol{x}_*) = 28.$$

$\mathcal{D}_1$ and $\mathcal{D}_2$ are two training samples.
$\widehat{f}_{\mathcal{D}_j}$ denotes the fitted tree from sample j.

Main idea

A prediction is unstable when it changes substantially across training samples.

We would like to keep the useful signal without depending on one sample.

Ensemble Strategies

Let B be the number of fitted trees, and let $\widehat{f}_b(\boldsymbol{x})$ be tree b's prediction for a new input $\boldsymbol{x}$.

$$\boxed{\widehat{f}_1(\boldsymbol{x}), \ldots, \widehat{f}_B(\boldsymbol{x}) \longrightarrow \widehat{f}_{\text{ens}}(\boldsymbol{x}).}$$

Two approaches:

- **Bagging:** average trees trained on resampled data.
- **Boosting:** sequentially add corrections to a current model.

First, we study how to generate different versions of a tree.

Bagging

DSC5001 ▪ Semester A, 2026/27

Bootstrap Motivation

Collecting B independent new datasets would be expensive.
Instead, use the available training set $\mathcal{D}$ of n records.
A **bootstrap sample** $\mathcal{D}^{*(b)}$ contains n independent draws *with replacement* from $\mathcal{D}$, for $b = 1, \ldots, B$.

$$\boxed{\mathcal{D} \longrightarrow \mathcal{D}^{*(1)}, \ldots, \mathcal{D}^{*(B)}.}$$

Repeated and omitted cases make the fitted trees differ.
Next we calculate two resamples of our deliveries.

Bootstrap Samples

Our six route IDs are (A, B, C, D, E, F), one ID for each complete training record.
Two bootstrap samples of size $n = 6$ might draw:

$$\mathcal{D}^{*(1)} = (A, A, C, D, D, F),$$
$$\mathcal{D}^{*(2)} = (B, C, C, E, F, F).$$

Both have six draws, but only four *distinct* routes.
These different records will produce different leaf predictions.

Bootstrap Counts

Route	A	B	C	D	E	F	Total
Sample 1	2	0	1	2	0	1	6
Sample 2	0	1	2	0	1	2	6

Let N_i^* be the number of copies of original record i in one size-n bootstrap sample.

$$\boxed{N_i^* \sim \text{Binomial}(n, 1/n), \quad \mathbb{E}[N_i^*] = 1.}$$

An expected count of one does *not* imply exactly one copy.
Now fit one simple tree to each resample.

First Bootstrap Tree

For transparent arithmetic, **hold the distance split at $d \leq 3.5$ km** in both bootstrap trees. The split is an illustrative fixed rule, not a newly optimized split.

Sample 1	**Near leaf**	**Far leaf**
Delivery times	$(12, 12, 17)$	$(22, 22, 29)$
Mean time	$41/3$	$73/3$

For a target route at $d_* = 4.5$ km, the first fitted tree predicts

$$\boxed{\widehat{f}^{*(1)}(d_*) = \frac{73}{3} \approx 24.33 \text{ min.}}$$

Here $\widehat{f}^{*(1)}$ is the tree fitted to $\mathcal{D}^{*(1)}$.

Second Bootstrap Tree

Apply the *same fixed split* $d \leq 3.5$ km to $\mathcal{D}^{*(2)}$.

Sample 2	**Near leaf**	**Far leaf**
Delivery times	$(14, 17, 17)$	$(26, 29, 29)$
Mean time	16	28

For the same target distance $d_* = 4.5$ km:

$$\boxed{\widehat{f}^{*(2)}(d_*) = 28 \text{ min.}}$$

Different bootstrap weights produce different predictions despite an identical split rule.

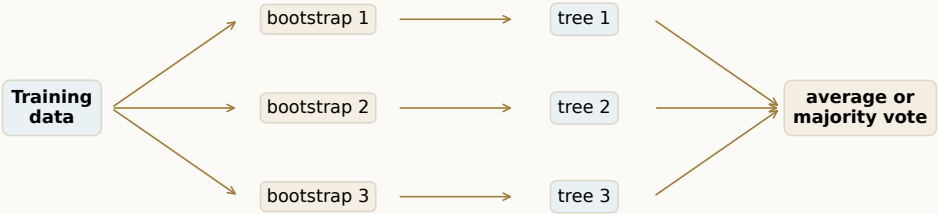
Bagged Prediction

In our two-tree example, $B = 2$ and the predictions for $d_* = 4.5$ km are $73/3$ and 28 minutes.

$$\begin{aligned}\widehat{f}_{\text{bag}}(d_*) &= \frac{\widehat{f}^{*(1)}(d_*) + \widehat{f}^{*(2)}(d_*)}{2} \\ &= \frac{1}{2}\left(\frac{73}{3} + 28\right) \\ &= \boxed{\frac{157}{6} \approx 26.17 \text{ min}}.\end{aligned}$$

The prediction is less tied to either individual resample.
The same operation extends to many bootstrap trees.

Bagging Workflow



Follow left to right: each resample trains its own tree; only the final *predictions* are averaged. The trees are fitted separately.

Bagging Formula

For B bootstrap trees, where tree b is fitted to $\mathcal{D}^{*(b)}$:

$$\boxed{\widehat{f}_{\text{bag}}(\boldsymbol{x}) = \frac{1}{B}\sum_{b=1}^{B} \widehat{f}^{*(b)}(\boldsymbol{x}).}$$

- $\boldsymbol{x}$: a new feature vector (distance, congestion, rain).
- b: tree index, $1 \le b \le B$.
- $\widehat{f}^{*(b)}(\boldsymbol{x})$: tree b's predicted delivery time.

Averaging becomes most useful when the predictions vary across samples.

Averaging Variance

Fix one input $\boldsymbol{x}_*$.
Let $Z_b = \widehat{f}_b(\boldsymbol{x}_*)$ denote the random prediction from tree b across repeated training samples.
Assume all B predictions have variance v and are mutually independent.
Define their average $\overline{Z}_B = B^{-1} \sum_{b=1}^{B} Z_b$.

$$\boxed{\mathrm{Var}(\overline{Z}_B) = \frac{1}{B^2} \sum_{b=1}^{B} \mathrm{Var}(Z_b) = \frac{v}{B}.}$$

For $v = 16$ and $B = 4$, the average variance is 4.
Their prediction errors can still be correlated across possible original datasets, even if bootstrap draws are generated independently.

Correlated Errors

Keep Z_b and $\overline{Z}_B$ from the previous slide.
Suppose every tree has variance v and any two distinct trees have correlation ρ:

$$\mathrm{Cov}(Z_b, Z_c) = \rho v, \qquad b \neq c.$$

There are B variance terms v and $B(B-1)$ covariance terms ρv:

$$\begin{aligned}\mathrm{Var}(\overline{Z}_B) &= \frac{Bv + B(B-1)\rho v}{B^2} \\ &= \boxed{v\left[\rho + \frac{1-\rho}{B}\right]}.\end{aligned}$$

For fixed positive ρ, increasing B cannot reduce variance below the limiting value $v\rho$.
This limitation will motivate random forests.

Correlation Example

Take $B = 4$ trees, variance $v = 16$ per tree.

Pairwise ρ	Calculation	**Average variance**
0	$16(0 + 1/4)$	4
0.25	$16(0.25 + 0.75/4)$	$\boxed{7}$
1	$16(1)$	16

Reducing similarity among tree errors can make averaging more effective than merely increasing B.

Classification Aggregation

Bagging also handles classification.
In a hypothetical delay task, $Y = 1$ means late delivery.
Let p_b be tree b's estimated delay probability for one package.

$$(p_1, \ldots, p_5) = (0.90, 0.70, 0.55, 0.20, 0.10).$$

Aggregation rule	**Decision at threshold** 0.5
Vote on each tree's class	$3/5$ vote delayed $\Rightarrow 1$
Average tree probabilities	$\overline{p} = 2.45/5 = 0.49 \Rightarrow 0$

Both are valid aggregation choices; they need not agree.
Next, we need to evaluate the ensemble without scoring on its own fitted cases.

OOB Motivation

One bootstrap tree uses only its selected records for fitting.
A record it never saw is **out of bag (OOB)** for that tree.
For $\mathcal{D}^{*(1)} = (A, A, C, D, D, F)$, the omitted route IDs are B and E.

Fit on selected cases $\rightarrow$ evaluate on omitted cases.

OOB predictions provide validation-like information without a separate holdout for each tree.

OOB Probability

Let q_n be the probability that one particular original record is missed by all n bootstrap draws. Each draw misses it with probability $1 - 1/n$.

$$\boxed{q_n = \left(1 - \frac{1}{n}\right)^n.}$$

For our $n = 6$ deliveries,

$$q_6 = (5/6)^6 \approx \boxed{0.335}.$$

As $n \to \infty$, $q_n \to e^{-1} \approx 0.368$.
Thus "36.8% OOB" is a large-sample approximation, not a fixed count in every resample.

OOB Calculation

Tree 1 omitted routes B and E.
Use its previously computed leaf means.

OOB route	Observed time y	Tree prediction $\widehat{y}$
B (2 km)	14	$41/3$
E (5 km)	26	$73/3$

The mean squared error (MSE) averages the squared prediction errors:

$$\boxed{\mathrm{MSE}_{\mathrm{OOB},1} = \frac{(14 - 41/3)^2 + (26 - 73/3)^2}{2} = \frac{13}{9} \approx 1.444.}$$

This is Tree 1's OOB error, not the whole ensemble's OOB error.

OOB Ensemble Prediction

For training case i, define $\mathcal{B}_i$ as the *set of tree indices* whose bootstrap samples omitted that case.
If $|\mathcal{B}_i| > 0$, its OOB ensemble prediction is

$$\widehat{f}_{\mathrm{OOB}}(\boldsymbol{x}_i) = \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} \widehat{f}^{*(b)}(\boldsymbol{x}_i).$$

Here $|\mathcal{B}_i|$ counts eligible trees; $\boldsymbol{x}_i$ and y_i are the original input and observed time.
Compare these predictions with y_i across cases that have at least one OOB tree.

OOB Limits

Our first two resamples omit different route IDs:

$\mathcal{D}^{*(1)}$: B, E omitted; $\mathcal{D}^{*(2)}$: A, D omitted.

Route	A	B	C	D	E	F
Number of OOB trees	1	1	0	1	1	0

Routes C and F have *no* OOB prediction from these two trees.
More trees usually improve coverage.
OOB estimates help model development; keep the final test set untouched.

Random Forests

DSC5001 ▪ Semester A, 2026/27

Split Similarity

Bagging changes the *training rows*. Why might its trees still make similar errors?
In parcel delivery, distance may dominate other features, so many bagged trees can choose distance near the root.

Similar splits $\rightarrow$ correlated predictions.

Recall: higher tree correlation ρ weakens the variance reduction from averaging.
Random forests change which *features* each node can inspect.

Feature Sampling

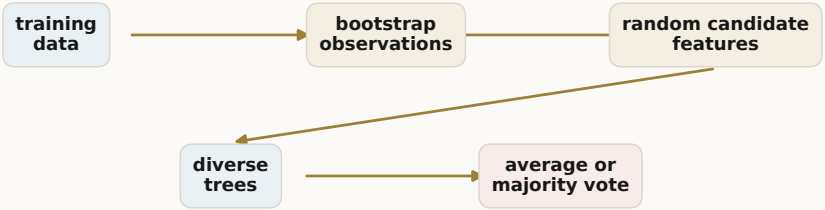
Our delivery dataset has $q = 3$ predictors: distance, congestion, and rain.
At each node, a random forest:

1. draws m_{try} distinct candidate features out of the q;
2. evaluates valid thresholds for those candidates;
3. chooses the best split *within the selected subset*.

$$\boxed{1 \leq m_{\text{try}} \leq q.}$$

The feature subset is drawn again at each node; the threshold itself is still optimized.

Forest Workflow



Two sources of randomness reduce similarity among trees and improve averaging.

Two different random choices: the bootstrap changes training *rows*; the per-node feature subset changes the *available split variables*.
Both help diversify trees.

Regression Split Criterion

For regression trees, choose the split with smaller total squared error.
For any node R, let $\overline{y}_R$ be the mean of its delivery times and define

$$\boxed{RSS(R) = \sum_{i \in R} (y_i - \overline{y}_R)^2.}$$

For a split into left and right children L, R, compare

$$\boxed{RSS(L) + RSS(R).}$$

This is the criterion for the next three feature comparisons.

Root RSS

For the six original delivery times,

$$y = (12, 14, 17, 22, 26, 29), \qquad \overline{y}_P = 20.$$

Here P is the parent node, $\overline{y}_P$ its mean, and RSS_P its sum of squared residuals:

$$\begin{aligned} RSS_P &= \sum_{i \in P} (y_i - \overline{y}_P)^2 \\ &= (-8)^2 + (-6)^2 + (-3)^2 + 2^2 + 6^2 + 9^2 \\ &= \boxed{230}. \end{aligned}$$

A good regression-tree split minimizes $RSS_L + RSS_R$, the sum for its left and right children.
To isolate feature selection, we keep these six rows fixed. A real forest additionally resamples training rows.

Distance Split

Split the parent at distance $d \leq 3.5$ km.
Each child predicts its mean.

	Near $(12, 14, 17)$	**Far** $(22, 26, 29)$
Mean	$43/3$	$77/3$
Child RSS	$38/3$	$74/3$

For example, near-child RSS is

$$\sum_{y \in \{12,14,17\}} (y - 43/3)^2 = 38/3.$$

$$\boxed{RSS_L + RSS_R = \frac{112}{3} \approx 37.33.}$$

The decrease from the parent is

$$230 - 112/3 = \boxed{578/3 \approx 192.67}.$$

Distance Thresholds

Distance has six distinct values, so there are five candidate cuts.
For each cut s, fit the two leaf means and add their RSS values:

Threshold s	1.5	2.5	3.5	4.5	5.5
Child RSS sum	153.20	83	$\mathbf{37.33}$	61.25	132.80

$s = 3.5$ km gives the smallest distance-split RSS.

The forest still needs to choose which features are available at a node.

Congestion Split

Now split on congestion $c = 0$ versus $c = 1$.

	No congestion	**Congestion**
Delivery times	$(12, 17, 29)$	$(14, 22, 26)$
Mean	$58/3$	$62/3$
RSS	$458/3$	$224/3$

$$\boxed{RSS_L + RSS_R = \frac{682}{3} \approx 227.33.}$$

Gain $= 230 - 682/3 = \boxed{8/3 \approx 2.67}$.

Congestion barely separates delivery times in these six records.

Rain Split

Finally split on rain $w = 0$ versus $w = 1$.

	No rain	**Rain**
Delivery times	$(12, 14, 29)$	$(17, 22, 26)$
Mean	$55/3$	$65/3$
RSS	$518/3$	$122/3$

$$\boxed{RSS_L + RSS_R = \frac{640}{3} \approx 213.33.}$$

Gain $= 230 - 640/3 = \boxed{50/3 \approx 16.67}$.

Rain improves this split more than congestion but much less than distance.

Split Comparison

The parent RSS is 230 for all three candidates.

Feature used	Child RSS	RSS decrease
Distance	37.33	192.67
Congestion	227.33	2.67
Rain	213.33	16.67

A standard CART node with all three candidates chooses **distance**.
If a forest node receives only *congestion and rain*, it chooses **rain** instead.
This is the diversity mechanism.

Random Feature Sets

Take $q = 3$ total features and $m_{\text{try}} = 2$ sampled features per node.

Random candidate subset	**Best available feature**
Distance + congestion	distance
Distance + rain	distance
Congestion + rain	rain

The three choices differ even if the node's data are identical.
Next we quantify the chance that the strongest feature is unavailable.

Feature Inclusion

Select m_{try} distinct features uniformly from q available features.
For any one named feature,

$$\boxed{\Pr(\text{included}) = \frac{m_{\text{try}}}{q}, \quad \Pr(\text{excluded}) = 1 - \frac{m_{\text{try}}}{q}.}$$

In our case, $q = 3$ and $m_{\text{try}} = 2$:

$$\boxed{\Pr(\text{distance excluded}) = \frac{1}{3}.}$$

Other predictors get opportunities to form splits in about one-third of sampled subsets.

Forest Prediction

Forest training uses random feature subsets, but regression predictions are still averaged. Let $B = 4$ fitted forest trees predict $(24, 26, 27, 29)$ minutes for the same new route $\boldsymbol{x}_*$.

$$\widehat{f}_{\mathrm{RF}}(\boldsymbol{x}_*) = \frac{1}{B}\sum_{b=1}^{B}\widehat{f}_b(\boldsymbol{x}_*) = \frac{24 + 26 + 27 + 29}{4} = 26.5.$$

The change from ordinary bagging lies in *tree construction*, not the aggregation formula.

Forest Settings

Setting	Role
B: number of trees	averaging stability and computation
m_{try}: candidate features per split	tree diversity versus split quality
Maximum depth / minimum leaf size	complexity of each tree

With $m_{\text{try}} = q$, feature subsampling disappears and the usual forest split rule becomes ordinary bagging.

Permutation Importance

After fitting the forest, estimate which feature affects prediction quality:

1. measure validation **RMSE** (root mean squared error; lower is better);
2. shuffle one feature across validation cases;
3. measure how much RMSE increases.

Feature shuffled	**RMSE after**	**Increase from** 4.2
Distance	7.1	2.9
Rain	4.4	0.2

Permutation importance measures *predictive dependence*, not a causal effect.

Random forests address instability; next we address errors that averaging does not fix.

Gradient Boosting

DSC5001 ▪ Semester A, 2026/27

Shared Prediction Error

Bagging and forests are designed mainly to stabilize trees.
But some errors may be shared.
Suppose four trees all predict 14 minutes for a delivery taking 18 minutes:

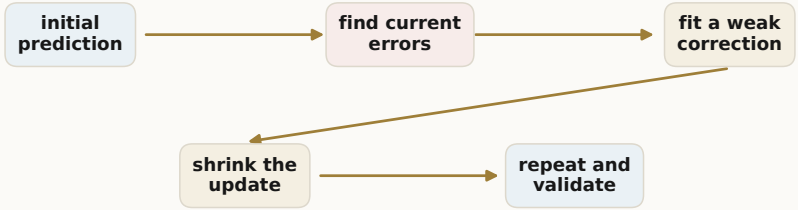
$$\frac{14 + 14 + 14 + 14}{4} = \boxed{14}.$$

Averaging has not corrected the four-minute underprediction.

Main idea

Boosting instead adds a new model that tries to correct the remaining error.

Boosting Workflow



Boosting is sequential: each new learner depends on the current ensemble.

Follow the arrows: calculate remaining errors → fit a small correction tree → update predictions. The next tree sees the *updated* errors, not the original ones.

Delivery-Time Example

For hand calculations, use a **second, smaller set of four parcel deliveries** from the same setting.
These are not the six routes used for the forest example.

Distance x_i (km)	1	2	3	4
Observed time y_i (min)	12	14	18	20

Here $i = 1, \ldots, 4$ indexes deliveries and x_i is a scalar distance.
We use **regression stumps**: trees with just one split and two constant predictions.
Each stage will correct the preceding predictions.

Initial Fit

A squared-error model begins with the constant minimizing training loss: the mean delivery time.

$$\boxed{F_0(x) = \overline{y} = \frac{12 + 14 + 18 + 20}{4} = 16.}$$

Write $\boldsymbol{y} = (12, 14, 18, 20)^\top$.
Define $e_{i,0} = y_i - F_0(x_i)$ and

$$\boldsymbol{e}_0 = (e_{1,0}, \ldots, e_{4,0})^\top.$$

Then

$$\boxed{\boldsymbol{e}_0 = (-4, -2, 2, 4).}$$

The initial **residual sum of squares** is

$$\boxed{RSS_0 = \sum_{i=1}^{4} e_{i,0}^2 = 40.}$$

Squared-Loss Gradient

Why fit residuals rather than the original responses?
For one observed time y and current prediction F, use the half-squared loss

$$\ell(y, F) = \frac{1}{2}(y - F)^2.$$

Its negative derivative with respect to the prediction is

$$\boxed{-\frac{\partial \ell(y, F)}{\partial F} = y - F.}$$

Thus the residual is exactly the **loss-reducing direction** for squared error.
The next small tree tries to approximate this direction.

Stump Thresholds

The first stump fits residuals $(-4, -2, 2, 4)$ at distances $(1, 2, 3, 4)$.
At each candidate cut, each leaf predicts its mean residual.

Cut s	1.5	2.5	3.5
Within-leaf residual RSS	$56/3$	$\mathbf{4}$	$56/3$

For $s = 2.5$, the two leaf means are -3 and 3:

$$RSS_{\text{stump}} = (-4+3)^2 + (-2+3)^2 + (2-3)^2 + (4-3)^2 = 4.$$

Choose $s = 2.5$ for the first correction tree.

First Residual Stump

Fit a one-split tree h_1 to the initial residual vector $e_0 = (-4, -2, 2, 4)$.
Use the selected distance threshold $x \leq 2.5$ km:

	$x \leq 2.5$	$x > 2.5$
Residuals	$(-4, -2)$	$(2, 4)$
Mean correction	-3	3

The fitted first correction tree is

$$\boxed{h_1(x) = \begin{cases} -3, & x \leq 2.5, \\ 3, & x > 2.5. \end{cases}}$$

Its leaf outputs are the means of the residuals reaching each leaf.

First Update

Let $\nu \in (0, 1]$ be the **learning rate**.
Use $\nu = 0.5$ to apply only half the first correction.

$$\boxed{F_1(x) = F_0(x) + \nu h_1(x).}$$

Distance x	1	2	3	4
Initial F_0	16	16	16	16
Correction $0.5h_1$	-1.5	-1.5	1.5	1.5
New prediction F_1	14.5	14.5	17.5	17.5

The update moves predictions toward observed times without applying the full correction.

First Loss Reduction

Define the new residuals $e_{i,1} = y_i - F_1(x_i)$.

$$\boxed{\boldsymbol{e}_1 = (-2.5, -0.5, 0.5, 2.5).}$$

Their sum of squares is

$$\boxed{RSS_1 = \sum_{i=1}^{4} e_{i,1}^2 = 2.5^2 + 0.5^2 + 0.5^2 + 2.5^2 = 13.}$$

Training RSS fell from 40 to 13.
The remaining pattern motivates the **second** correction tree.

Second Residual Stump

The second stump h_2 is fitted to the *new* residual vector $\boldsymbol{e}_1$, not the original labels.

$$\boldsymbol{e}_1 = (-2.5, -0.5, 0.5, 2.5).$$

Using split $x \leq 2.5$ km, its leaf means are

$$\overline{e}_L = \frac{-2.5 - 0.5}{2} = -1.5,$$

$$\overline{e}_R = \frac{0.5 + 2.5}{2} = 1.5.$$

Therefore,

$$\boxed{h_2(x) = \begin{cases} -1.5, & x \leq 2.5, \\ 1.5, & x > 2.5. \end{cases}}$$

Second Update

Apply the second correction with the same learning rate $\nu = 0.5$:

$$\boxed{F_2(x) = F_1(x) + 0.5h_2(x).}$$

Distance x	1	2	3	4
$F_1(x)$	14.5	14.5	17.5	17.5
$0.5h_2(x)$	-0.75	-0.75	0.75	0.75
$F_2(x)$	13.75	13.75	18.25	18.25

F_2 is one cumulative model, not an average of the two stumps.

Second RSS Reduction

The second residual vector has entries $e_{i,2} = y_i - F_2(x_i)$:

$$\boxed{\boldsymbol{e}_2 = (-1.75, 0.25, -0.25, 1.75).}$$

$$\boxed{RSS_2 = \sum_i e_{i,2}^2 = 1.75^2 + 0.25^2 + 0.25^2 + 1.75^2 = 6.25.}$$

Stage	Initial	After first tree	After second tree
Training RSS	40	13	6.25

The stagewise corrections improve training fit; new-data performance still needs validation.

Two-Stage Prediction

The resulting two-stage regression model is

$$F_2(x) = 16 + 0.5h_1(x) + 0.5h_2(x).$$

For a *new* delivery at distance $x_* = 2.3$ km:

$$\boxed{F_2(2.3) = 16 - 1.5 - 0.75 = 13.75 \text{ min.}}$$

For $x_* = 3.7$ km:

$$\boxed{F_2(3.7) = 16 + 1.5 + 0.75 = 18.25 \text{ min.}}$$

This is an **additive prediction**: start from F_0 and apply both learned corrections.

Learning-Rate Comparison

Keep the first stump fixed at $h_1 = (-3, -3, 3, 3)$ on the four training distances.
Vary only the learning rate ν.

ν	Pred. $x \leq 2.5$	Pred. $x > 2.5$	Training RSS
0.25	15.25	16.75	24.25
0.50	14.50	17.50	13.00
1.00	13.00	19.00	4.00

A larger first step reduces training RSS more *here*, but repeated large steps can become too sensitive to noise.
Smaller ν often needs more stages to achieve a comparable fit.

Weak Learner Size

The first stump had two leaves and could not fit every individual residual.
A much deeper tree might fit all four residuals in one stage, but in noisy data that risks memorizing errors.

	Shallow correction	**Deep correction**
Per-stage flexibility	limited	high
Noise sensitivity	often lower	can be higher

Boosting therefore controls complexity through **tree size**, **learning rate**, and **number of rounds**.

General Gradient Boosting

Squared-error boosting fits ordinary residuals.
For other losses, the needed correction may differ.
Let $\ell(y_i, F)$ be the differentiable loss for case i at a predicted score F.
At stage m, $F_{m-1}(x_i)$ is the current score.

$$r_{i,m} = -\left.\frac{\partial \ell(y_i, F)}{\partial F}\right|_{F=F_{m-1}(x_i)}.$$

$r_{i,m}$ is the **negative loss gradient**, or pseudo-residual, for case i at stage m.
Fit the next small tree to these values.

Boosting Update

At stage m, fit a small correction tree $h_m(x)$ to the negative gradients $r_{i,m}$.
Update the model by

$$\boxed{F_m(x) = F_{m-1}(x) + \nu\gamma_m h_m(x).}$$

- m: stage index; F_m: accumulated model after stage m.
- h_m: correction tree; γ_m: fitted step multiplier.
- ν: learning rate (shrinkage), already used in our example.

Our regression example used $\gamma_1 = \gamma_2 = 1$.
Classification will reuse the same framework with a new loss.

Boosted Classification

DSC5001 ▪ Semester A, 2026/27

Delivery-Delay Labels

Return to the **four-delivery teaching sample**.
We now predict delay status, not minutes.

Distance x_i (km)	1	2	3	4
Time t_i (min)	12	14	18	20
Delay label y_i	0	0	1	1

Here

$$y_i = \mathbf{1}\{t_i > 16\},$$

where $\mathbf{1}\{\cdot\}$ is 1 when its condition holds and 0 otherwise.
Restart boosting from the beginning: F_m will now mean a *log-odds score*, not a predicted time.

Logistic Scores

For a new distance x, let $F(x)$ be the fitted **log-odds score**.
Convert it to a model probability of delay:

$$\boxed{p(x) = \Pr_{\text{model}}(Y = 1 \mid x) = \sigma(F(x)) = \frac{1}{1 + e^{-F(x)}}.}$$

Here σ is the sigmoid function.

Score F	-1	0	1
Probability $\sigma(F)$	0.269	0.500	0.731

Boosting adds corrections to F, and the sigmoid keeps probabilities between 0 and 1.

Logistic Loss

For one case, $y \in \{0, 1\}$ is the delay label, F the log-odds score, and $p = \sigma(F)$ the estimated delay probability.
Cross-entropy as a function of F is

$$\boxed{\ell(y, F) = -y \log p - (1 - y) \log(1 - p).}$$

For an actual delay ($y = 1$), probabilities $p = 0.9$ and $p = 0.1$ give losses 0.105 and 2.303, respectively.
Wrong, confident probabilities cost much more.
We must correct *this loss*, not squared minutes.

Logistic Gradient

Recall $p = \sigma(F)$, where F is the current log-odds score.
Its derivative is

$$\frac{dp}{dF} = p(1-p).$$

By the chain rule,

$$\begin{aligned}\frac{\partial \ell}{\partial F} &= \left(-\frac{y}{p} + \frac{1-y}{1-p}\right) p(1-p) \\ &= \boxed{p - y}.\end{aligned}$$

Therefore the negative gradient for one case is

$$\boxed{r = y - p.}$$

This correction is a *label minus predicted probability*, not a time residual.

Initial Classification Fit

For the four binary labels $\boldsymbol{y} = (0, 0, 1, 1)$, the positive-class proportion is $2/4 = 0.5$. Initialize the common log-odds score at

$$\boxed{F_0 = \log \frac{0.5}{0.5} = 0.}$$

Thus the initial probability vector is

$$\boldsymbol{p}^{(0)} = (0.5, 0.5, 0.5, 0.5).$$

The negative-gradient vector in distance order is

$$\boxed{\boldsymbol{r}^{(0)} = \boldsymbol{y} - \boldsymbol{p}^{(0)} = (-0.5, -0.5, 0.5, 0.5).}$$

First Logistic Stump

Fit a fresh stump to the *classification pseudo-residuals* $\boldsymbol{r}^{(0)}$.

	$x \le 2.5$	$x > 2.5$
Pseudo-residuals	$(-0.5, -0.5)$	$(0.5, 0.5)$
Stump output	-0.5	0.5

The first classification correction tree is

$$\boxed{h_1(x) = \begin{cases} -0.5, & x \le 2.5, \\ 0.5, & x > 2.5. \end{cases}}$$

Here h_1 is *new*; it is not the regression stump used earlier.

First Logistic Update

For this simplified demonstration, set the tree step size $\gamma_1 = 1$ and learning rate $\nu = 0.5$.

$$F_1(x) = F_0(x) + \nu\gamma_1 h_1(x).$$

On the four distances, the new score vector is

$$\boldsymbol{F}^{(1)} = (-0.25, -0.25, 0.25, 0.25).$$

Applying the sigmoid componentwise gives

$$\boxed{\boldsymbol{p}^{(1)} \approx (0.438, 0.438, 0.562, 0.562).}$$

Negative cases move below 0.5; positive cases move above it.

Logistic Loss Reduction

Let L_m be the **sum** of logistic losses over the four cases after boosting stage m.
Before the first stump,

$$\boxed{L_0 = 4 \log 2 \approx 2.773.}$$

After the first stump, each observed label receives probability $\sigma(0.25)$, so

$$\boxed{L_1 = 4 \log(1 + e^{-0.25}) \approx 2.304.}$$

The model assigns more probability to the observed labels, decreasing cross-entropy.

Second Logistic Stump

After Stage 1, the score for a positive case is $F_1 = 0.25$, giving

$$p_1 = \sigma(0.25) \approx 0.5622, \qquad 1 - p_1 \approx 0.4378.$$

The four updated negative gradients are

$$\boxed{\boldsymbol{r}^{(1)} = \boldsymbol{y} - \boldsymbol{p}^{(1)} \approx (-0.4378, -0.4378, 0.4378, 0.4378).}$$

With the same two-leaf split, $\nu = 0.5$, and $\gamma_2 = 1$:

$$\boxed{F_2(3) = 0.25 + 0.5(0.4378) \approx 0.4689.}$$

The first two scores are -0.4689 and the last two are $+0.4689$.

Second Logistic Loss

Apply the sigmoid to the four updated scores:

$$\boxed{\boldsymbol{p}^{(2)} \approx (0.3849, 0.3849, 0.6151, 0.6151).}$$

The probability assigned to each observed class is about 0.6151.
Thus the total cross-entropy after Stage 2 is

$$\boxed{L_2 = 4\log(1 + e^{-0.4689}) \approx 1.944.}$$

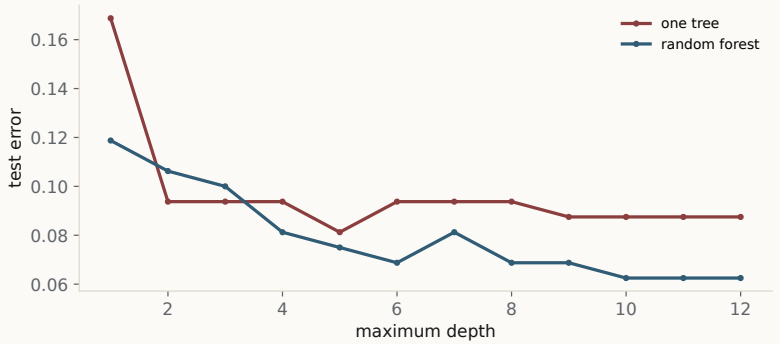
Stage	Initial	First	Second
Total loss L_m	2.773	2.304	1.944

These are illustrative gradient steps; some implementations optimize leaf-step sizes differently.

Model Selection

DSC5001 ▪ Semester A, 2026/27

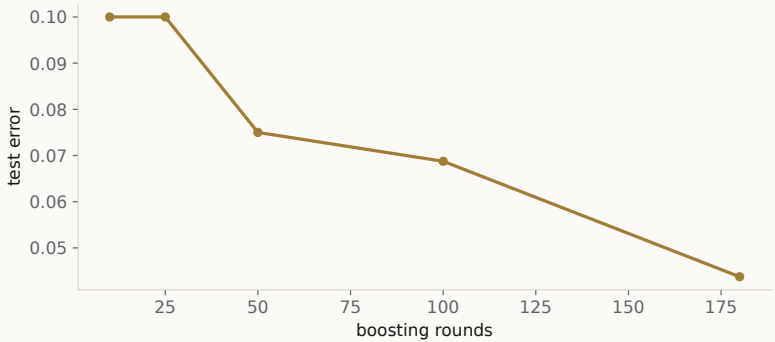
Depth and Test Error



Read the curves against depth: averaging can make forests less sensitive to an unstable individual tree.

The plot illustrates behavior; use *validation* rather than final-test inspection to choose depth.

Boosting Rounds



Read left to right: early corrections often help, but too many stages may fit noise. Select the number of stages using validation performance.

Round Selection

Let M be the total number of boosting rounds, one newly added tree per round.
Suppose held-out **RMSE** (root mean squared error; lower is better) is:

M	10	30	50	70	90
Validation RMSE	2.8	1.9	**1.6**	1.8	2.1

The lowest error among these checkpoints is at

$$\boxed{M = 50 \text{ rounds.}}$$

Save that stage for final testing; do not repeatedly tune on test data.

Early Stopping

During boosting, monitor an error metric on held-out validation data.

- Save the stage with the lowest validation error.
- Stop if later stages fail to improve for a specified patience period.
- Retain the selected stage, not automatically the last stage.

Early stopping controls how many correction trees are retained.

Model Comparison

Compare delivery-time models using the *same* training and validation split.
RMSE is root mean squared prediction error (lower is better).

Method	Training RMSE	Validation RMSE
Bagging	1.20	2.10
Random forest	1.10	**1.82**
Boosting	**0.80**	1.95

These are illustrative values.
Boosting fits training data best, but the forest has the lowest *validation* RMSE.
Choice: the forest, followed by evaluation on an untouched test set.

Evaluation Roles

Data	Purpose
Training	fit trees and boosting corrections
OOB / validation	select ensemble settings without using test labels
Final test	assess the completed selected procedure

Repeatedly tuning on final test results makes the test estimate optimistic.

Boosting Software

XGBoost and **LightGBM** implement efficient boosted-tree training with additional optimization and regularization features.
The same core questions remain:

- How large should individual trees be?
- How much should each new tree change predictions?
- How many boosting stages should be retained?

Use validation and careful evaluation to answer them.

Summary

Bagging	stabilize tree predictions by bootstrap averaging
Random forest	reduce tree similarity with random split features
Gradient boosting	correct the current model using loss gradients
Evaluation	select the full procedure on held-out data

Different sources of error call for different ensemble strategies.

Week 10: bagging, random forests, and boosting

Questions and discussion